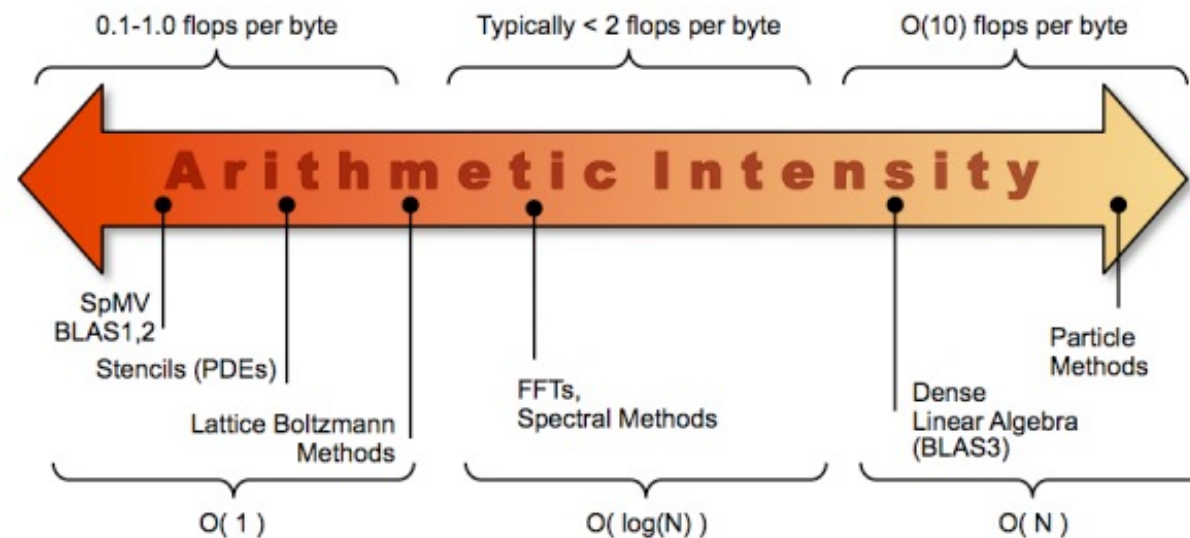


# Parallel Graphs Algorithms

Parallel Algorithms

João Luís Sobral (jls@di ...)

March/2026



## Data storage

Regular : vector, matrix

Irregular (pointer based): **tree, graphs**

**Data accesses**

- Regular / **irregular**

# Parallel Algorithms & data storage

recap: efficient matrix multiplication

BLIS on  
A64FX



- BLIS implementation

- Multiple levels of tiling
  - explores temporal locality

```

for  $j_c = 0, \dots, n-1$  in steps of  $n_c$ 
  for  $p_c = 0, \dots, k-1$  in steps of  $k_c$ 
    for  $i_c = 0, \dots, m-1$  in steps of  $m_c$ 
      for  $j_r = 0, \dots, n_c-1$  in steps of  $n_r$ 
        for  $i_r = 0, \dots, m_c-1$  in steps of  $m_r$ 
           $C(i_r:i_r+m_r-1, j_r:j_r+n_r-1) \pm \dots$ 
        endfor
      endfor
    endfor
  endfor
endfor
    
```

- (runtime) packing of A & B

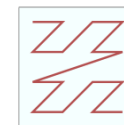
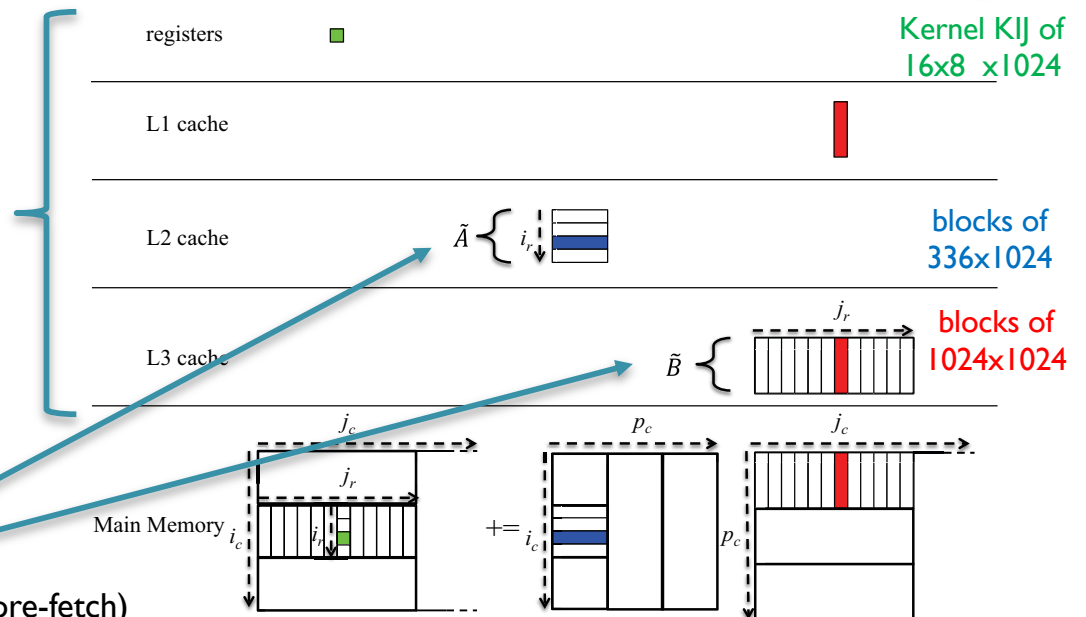
- improves spatial locality (& data pre-fetch)
- improves virtual memory temporal locality (4 KiB VM pages with TLB)

- Alignment & padding: spatial locality?

- Tuning for cache line size & cache replacement policy

- Alternative matrix storage (example):

- Z-order or Morton data layout

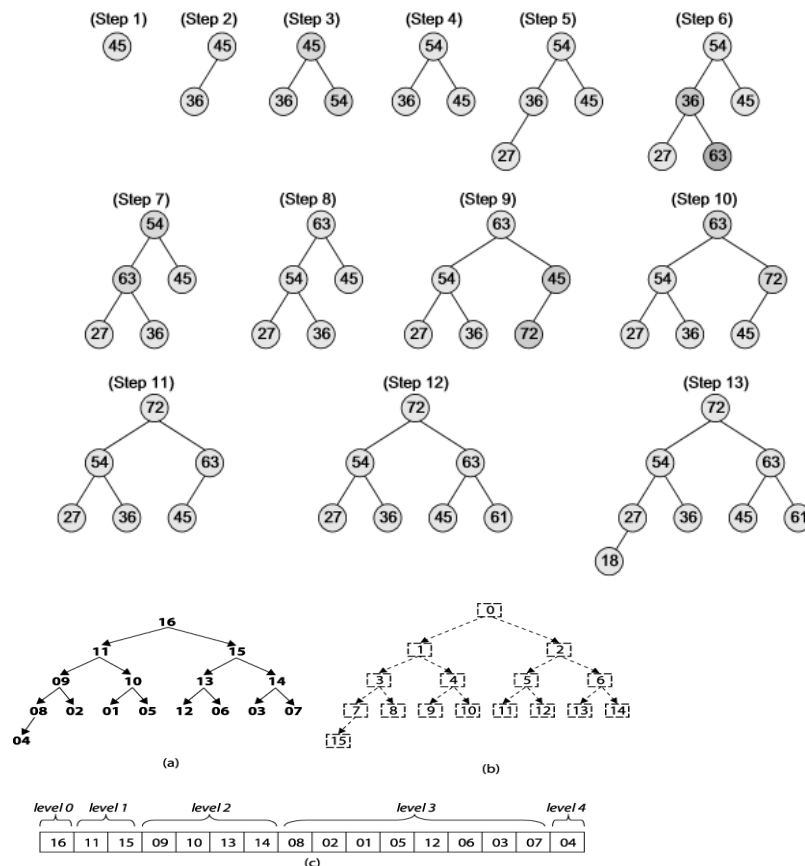


| +  | 0  | 1  | 4  | 5  | 16 | 17 | 20 | 21 |
|----|----|----|----|----|----|----|----|----|
| 0  | 0  | 1  | 4  | 5  | 16 | 17 | 20 | 21 |
| 2  | 2  | 3  | 6  | 7  | 18 | 19 | 22 | 23 |
| 8  | 8  | 9  | 12 | 13 | 24 | 25 | 28 | 29 |
| 10 | 10 | 11 | 14 | 15 | 26 | 27 | 30 | 31 |
| 32 | 32 | 33 | 36 | 37 | 48 | 49 | 52 | 53 |
| 34 | 34 | 35 | 38 | 39 | 50 | 51 | 54 | 55 |
| 40 | 40 | 41 | 44 | 45 | 56 | 57 | 60 | 61 |
| 42 | 42 | 43 | 46 | 47 | 58 | 59 | 62 | 63 |

# Parallel Algorithms & data storage

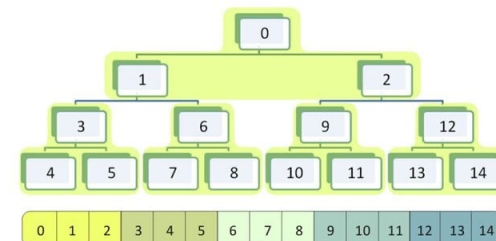
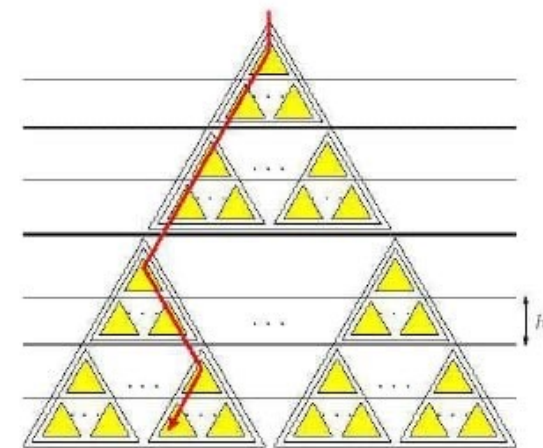
## recap: heap-sort

- heap-sort
  - (recap) Irregular data accesses (?)



## Alternative storage?

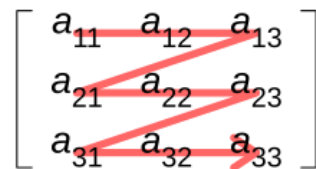
Example: Van Emde Boas Trees



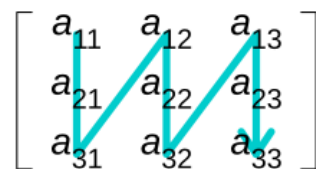
# Parallel Algorithms & data storage

- **Cache oblivious algorithms** (from Wikipedia)
  - “algorithms designed to take advantage of a processor cache without having the size of the cache (or the length of the cache lines, etc.) as an explicit parameter”
  - “Typically, work by a recursive divide-and-conquer algorithm, where the problem is divided into smaller and smaller subproblems”
  - “Simplest example: recursive matrix transpose:”
    - the matrices will continue to be divided recursively even after they fit into the cache

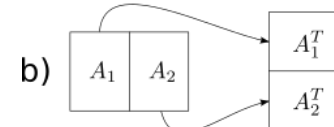
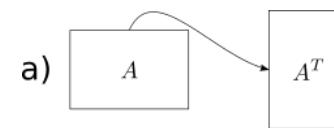
Row-major order



Column-major order



Cache misses  $O(N^2)$



Recursively  
transpose ...

Cache misses  $O(I + N^2/B)$

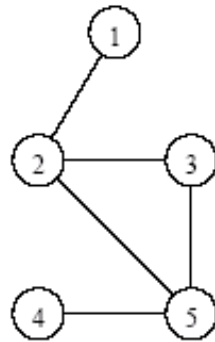
(same asymptotic complexity as a handcrafted)

# Graph Algorithms

- Base Concepts
  - A graph is a pair of  $(V, E)$ 
    - $V$  is a set of **Vertices**
    - $E$  is a set of **Edges**
  - Undirected graphs: edges are unordered pairs of vertices  $(u, v)$
  - Directed graphs: edges are ordered pairs of vertices  $(u, v)$
  - Weighted graph: each edge as a weight  $w$ , e.g.  $(u, v, w)$
  - Sparse graph:  $|E| \ll O(|V|^2)$
- Common graph representations
  - Adjacency matrix:  $a_{i,j} = 1$  iff  $(v_i, v_j) \in E$
  - Adjacency list
    - Array of  $|V|$  linked lists, each list contains one vertex adjacent vertices

# Graph Algorithms

- Adjacency matrix
  - $|V| \times |V|$  matrix
    - matrix element  $a_{i,j} = 1$  if nodes  $i$  and  $j$  share an edge; 0 otherwise
    - for a weighted graph,  $a_{i,j} = w_{i,j}$ , the edge weight
  - Requires  $\Theta(|V|^2)$  space



Undirected graph

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

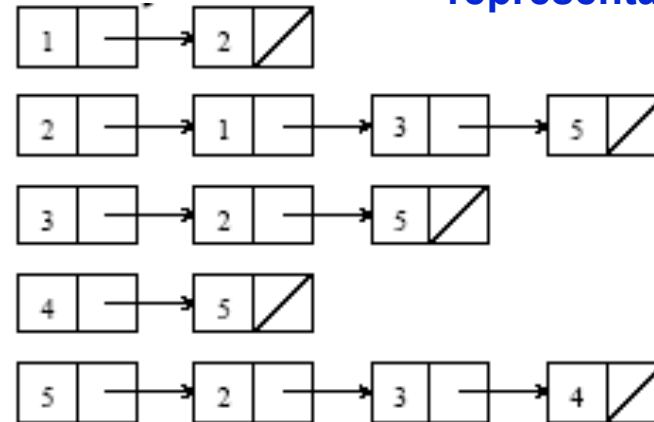
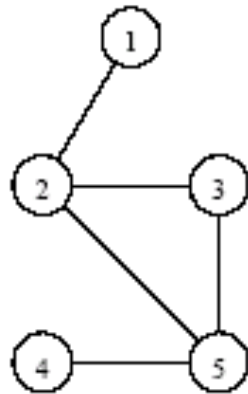
Adjacency matrix  
representation

adjacency matrix is  
symmetric about the diagonal

# Graph Algorithms

- Adjacency list
  - An array  $Adj[1..|V|]$  of lists
    - Each list  $Adj[v]$  is a list of all vertices adjacent to  $v$
  - Requires  $\Theta(|E|)$  space

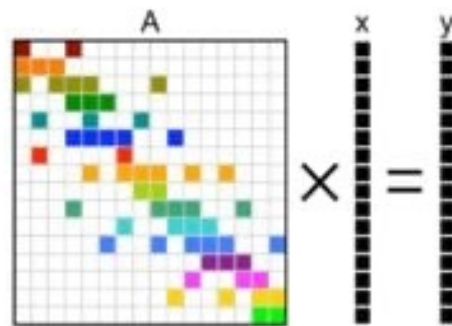
Adjacency list  
representation



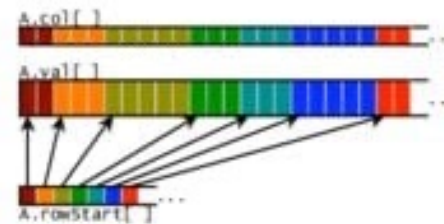
- More efficient representations:
  - Compressed sparse row (col), ELL, JDS (for GPUs) , etc...

# Sparse matrix representations

- Traditional storage: Compressed Sparse Row



(a)  
algebra conceptualization

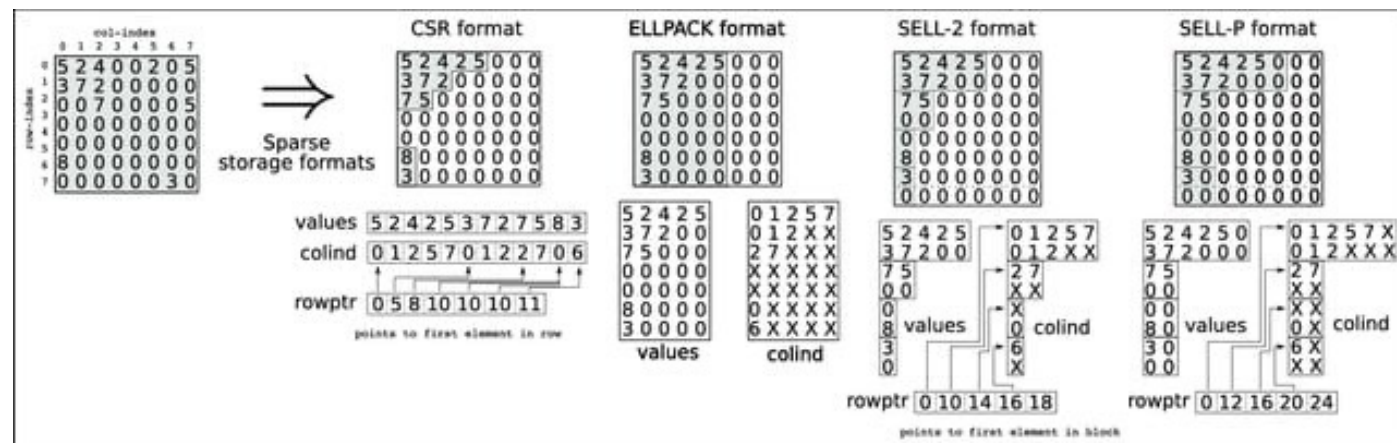


(b)  
CSR data structure

```
for (r=0; r<A.rows; r++) {
    double y0 = 0.0;
    for (i=A.rowStart[r]; i<A.rowStart[r+1]; i++){
        y0 += A.val[i] * x[A.col[i]];
    }
    y[r] = y0;
}
```

(c)  
CSR reference code

- Other formats



# Sparse matrix representations

- FVLib (example)

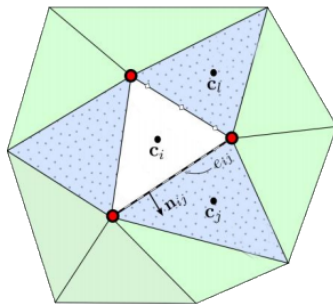


Figura 15.: Exemplo de uma malha.

```

1 for N in totalCells do
2   makeFlux(..., F, ...) {
3     for I in totalEdges do
4       if (!FRONTIER[I])
5         leftPhi = PHI[LEFT[I]];
6         rightPhi = PHI[RIGHT[I]];

```

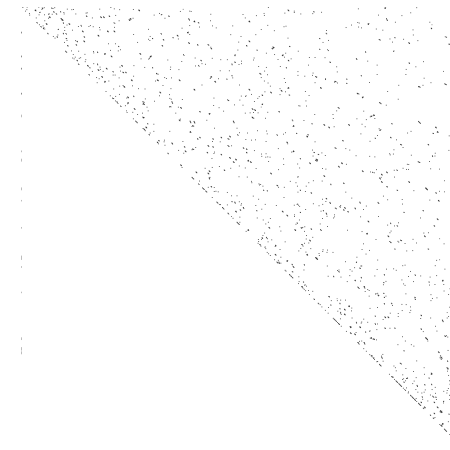


Figura 18.: Matriz original.

$$\sum F_{ij} = [V.n_{ij}]^+ \phi_i + [V.n_{ij}]^- \phi_j - k \frac{\phi_j - \phi_i}{|b_i.b_j|}$$

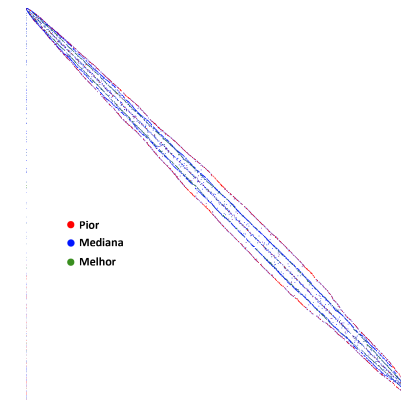
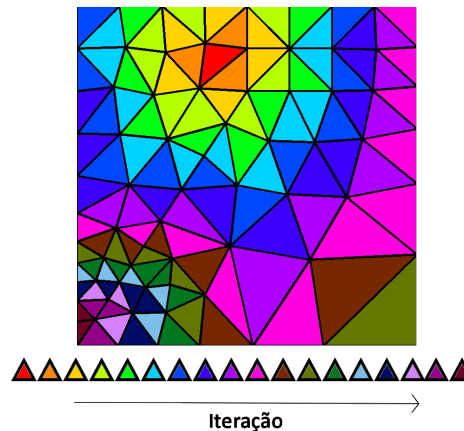
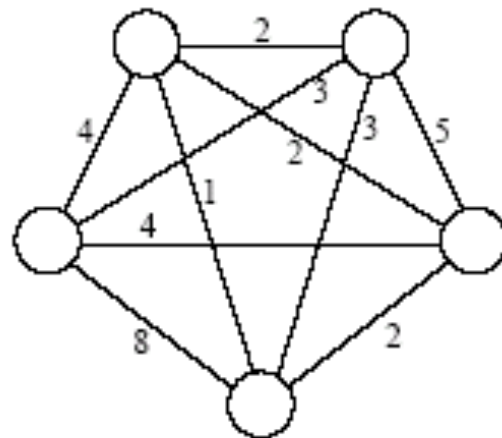


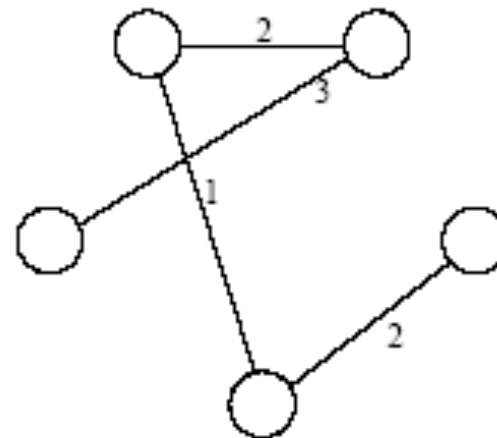
Figura 23.: Matriz de adjacência depois da renumeração.

# Graph Algorithms

- *Spanning tree* of a connected undirected graph  $G$ 
  - subgraph of  $G$  that is a tree containing all the vertices of  $G$
  - if graph is not connected: spanning forest
- *Weight* of a subgraph in a weighted graph
  - sum of the weights of the edges in the subgraph
- *Minimum spanning tree* (MST) for weighted undirected graph
  - spanning tree with minimum weight



Undirected graph



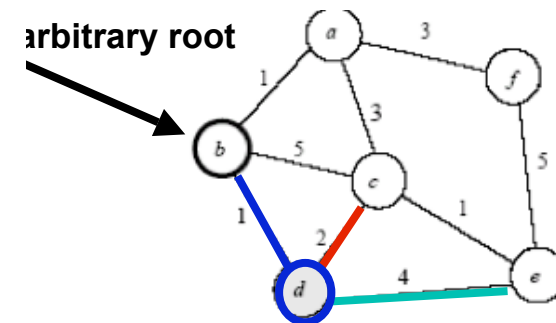
Minimum spanning tree

# Graph Algorithms

- Minimal Spanning Tree: Prim (  $O(V \log E)$  )
  - touch every vertex in the graph through the smallest weighted edges.

```

CurrentVertex = Random Start Vertex;
Neighbours    = Neighbours Of(Current Vertex);
Add To PQ(Neighbours);
while(Unvisited Vertices Exist) {
    // lazy removal of previously visited neighbours
    do {
        Min Neighbour = Get Minimum(PQ);
        Current Vertex = Vertex Of(MinNeighbour);
    } while (is Visited(Current Vertex));
    visit(Current Vertex);
    Add To MST(Min Neighbour);
    Neighbours = Neighbours Of(Current Vertex);
    // add unvisited neighbours to PQ
    for each neighbour in Neighbours {
        if(is Not Visited(Vertex Of(neighbour)))
            Add To PQ(neighbour);
    }
}
    
```



$O(\log_2 E)$  with a PQ



works by successively adding graph neighbour connections to the PQ and removing the smallest weight connection and marking the touched vertices as visited



$O(\log_2 E)$  with a PQ

# Graph Algorithms

- How to compute the **Minimal Spanning Tree in parallel?**
  - touch every vertex in the graph through the smallest weighted edges.

```
CurrentVertex = Random Start Vertex;  
Neighbours    = Neighbours Of(Current Vertex);  
Add To PQ(Neighbours);  
while(Unvisited Vertices Exist) {  
    // lazy removal of previously visited neighbours  
    do {  
        Min Neighbour = Get Minimum(PQ);  
        Current Vertex = Vertex Of(MinNeighbour);  
    } while (is Visited(Current Vertex));  
    visit(Current Vertex);  
    Add To MST(Min Neighbour);  
    Neighbours = Neighbours Of(Current Vertex);  
    // add unvisited neighbours to PQ  
    for each neighbour in Neighbours {  
        if(is Not Visited(Vertex Of(neighbour)))  
            Add To PQ(neighbour);  
    }  
}
```

- **Each iteration gets the minimum weight edge from the PQ, but that edge may be inserted in the current iteration**

# Graph Algorithms

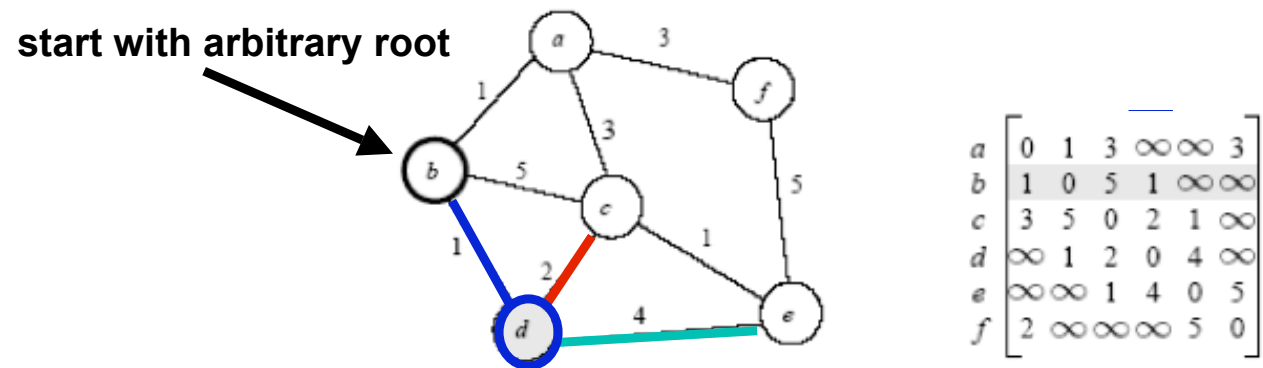
- Minimal Spanning Tree: Prim (formulation without PQ: adjacency matrix)  $O(V^2)$

```
1.  procedure PRIM_MST( $V, E, w, r$ )
2.  begin
3.       $V_T := \{r\}$ ; // initialize spanning tree vertices  $V_T$  with vertex  $r$ , the designated root
4.       $d[r] := 0$ ; // compute  $d[\cdot]$ , the
5.      for all  $v \in (V - V_T)$  do // weight between
6.          if edge  $(r, v)$  exists set  $d[v] := w(r, v)$ ; //  $r$  and each
7.          else set  $d[v] := \infty$ ; // vertex outside  $V_T$ 
8.      while  $V_T \neq V$  do // while there are vertices outside  $T$ 
9.          begin
10.             find a vertex  $u$  such that  $d[u] := \min\{d[v] | v \in (V - V_T)\}$ ;
11.              $V_T := V_T \cup \{u\}$ ; // add  $u$  to  $T$ 
12.             for all  $v \in (V - V_T)$  do // recompute  $d[\cdot]$  now
13.                  $d[v] := \min\{d[v], w(u, v)\}$ ; // that  $u$  is in  $T$ 
14.             endwhile
15.          end PRIM_MST
```

// use  $d[\cdot]$  to find  $u$ ,  
// vertex closest to  $T$

# Graph Algorithms

- Minimal Spanning Tree: Prim (formulation without PQ: adjacency matrix)

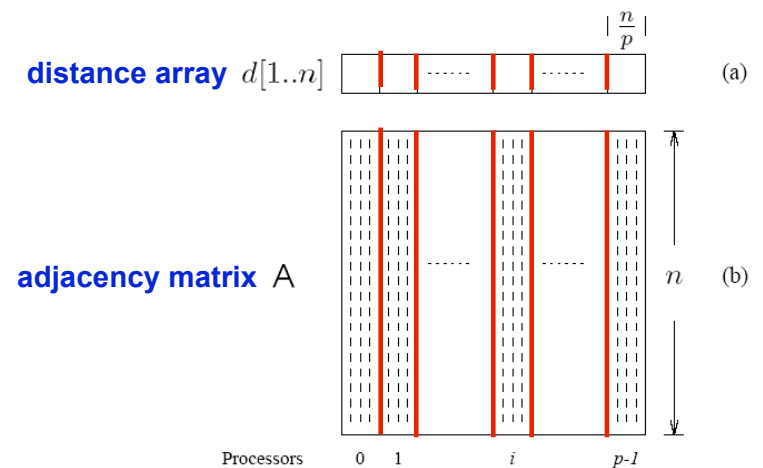


2 iterations of Prim's algorithm

# Graph Algorithms

- **Parallel Prim**

- outer loop ( $|V|$  iterations): hard to parallelize
  - adding 2 vertices to tree concurrently is problematic
- inner loop: relatively easy to parallelize
  - consider which vertex is closest to MST in parallel
- **Approach**
  - partition adjacency matrix in a 1-D block fashion (blocks of columns)
  - partition distance vector  $d$  accordingly
  - process first identifies the locally closest node
  - performs a global reduction to select globally closest node
  - leader inserts node into MST
  - broadcasts choice to all processes
  - each process updates its part of  $d$  vector locally based on choice



**partition  $d$  and  $A$  among  $p$  processes**

# Graph Algorithms

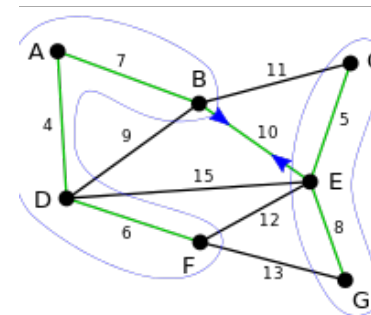
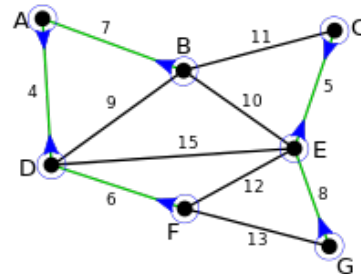
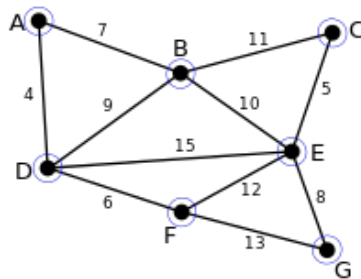
- Parallel Prim (cont.2)
  - Cost to select the minimum entry
    - $O(n/p)$ : scan  $n/p$  local part of  $d$  vector on each processor
    - $O(\log p)$  all-to-one reduction across processors
  - Broadcast next node selected for membership
    - $O(\log p)$
  - Cost of locally updating  $d$  vector
    - $O(n/p)$ : replace  $d$  vector with min of  $d$  vector and matrix row
- Total parallel time per iteration
  - $O(n/p + \log p)$
- Total parallel time
  - $O(n^2/p + n \log p)$

# Graph Algorithms

- Minimal Spanning Tree: Boruvka

- The algorithm begins by first examining each vertex and adding the cheapest edge from that vertex to another in the graph, without regard to already added edges, and continues joining these groupings in a like manner until a tree spanning all vertices is completed.

```
0 Input: A connected graph G whose edges have distinct weights
1 Initialize a forest T to be a set of one-vertex trees, one for each vertex of the graph.
2 While T has more than one component:
3   For each component C of T:
4     Begin with an empty set of edges S
5     For each vertex v in C:
6       Find the cheapest edge from v to a vertex outside of C, and add it to S
7     Add the cheapest edge in S to T
8   Combine trees connected by edges to form bigger components
9 Output: T is the minimum spanning tree of G.
```



# Graph Algorithms

## •Single Source Shortest Paths

- find the shortest paths from vertex  $v \in V$  to all other vertices in  $V$
- Dijkstra's algorithm: similar to Prim's algorithm
  - maintains a set of nodes for which the shortest paths are known
  - grows set by adding node closest to source using one of the nodes in the current shortest path set

```
1.  procedure DIJKSTRA_SINGLE_SOURCE_SP( $V, E, w, s$ )
2.  begin
3.     $V_T := \{s\};$  // initialize tree vertices  $V_T$  with vertex  $s$ , the designated src
4.    for all  $v \in (V - V_T)$  do // compute  $l[v]$ , the
5.      if  $(s, v)$  exists set  $l[v] := w(s, v);$  // weight between
6.      else set  $l[v] := \infty;$  //  $s$  and each vertex  $v \notin V_T$ 
7.    while  $V_T \neq V$  do // while some vertices are not in  $V_T$ 
8.      begin
9.        find a vertex  $u$  such that  $l[u] := \min\{l[v] | v \in (V - V_T)\};$ 
10.        $V_T := V_T \cup \{u\};$  // add  $u$  to  $T$ 
11.       for all  $v \in (V - V_T)$  do // recompute  $l[v]$ 
12.          $l[v] := \min\{l[v], l[u] + w(u, v)\};$  // now that  $u$  is in  $T$ 
13.       endwhile
14.     end DIJKSTRA_SINGLE_SOURCE_SP
```

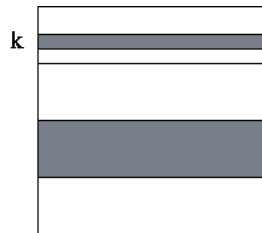
// use  $l[v]$  to find  $u$ ,  
// next vertex closest  
// src

# Graph Algorithms

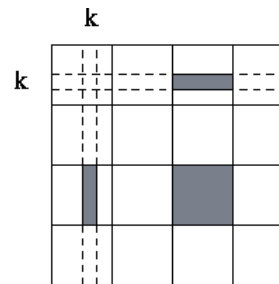
## • All-Pairs Shortest Paths ( $O(n^3)$ )

- find the shortest between all-pairs of vertices
- Parallel alternatives for Prim/Dijkstra's:
  - Source partitioned: Partition the vertices among processors
    - Embarrassing parallel (no communication required) => cost optimal ( $O(n^2)$ )
    - Limited to  $n$  processors
  - Source parallel: **use a parallel formulation of the shortest path problem to increase concurrency**
- Parallel Floyd's Algorithm
  - the cost of a path can be expressed by the sum of the costs through an intermediate vertex
    - for  $k, i, j$
  - One dimensional matrix partition VS two dimensional matrix partition

$$d_{i,j}^{(k)} = \min(d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)})$$



$$t_c \frac{N^3}{P} + N \log P (t_s + t_w N).$$



$$t_c \frac{N^3}{P} + N \log P \left( t_s + t_w \frac{N}{\sqrt{P}} \right)$$

# Graph Algorithms

- **Sparse graphs**

- Algorithms can be improved significantly if we make use of the sparseness
- Example: Prim's algorithm complexity
  - *can be reduced to  $O(|E| \log n)$*
  - *use heap to maintain costs*
  - *outperforms original as long as  $|E| = O(n^2 / \log n)$*
- Sparse algorithms: use adjacency list instead of matrix
- Partitioning adjacency lists is more difficult for sparse graphs
  - *How do we balance number of vertices or edges?*
- Parallel algorithms typically make use of graph structure or degree information for performance